

Group 8:

Cross-Sectional Stock Return Prediction Applying Neural Network

FINA 4713



Villvalin

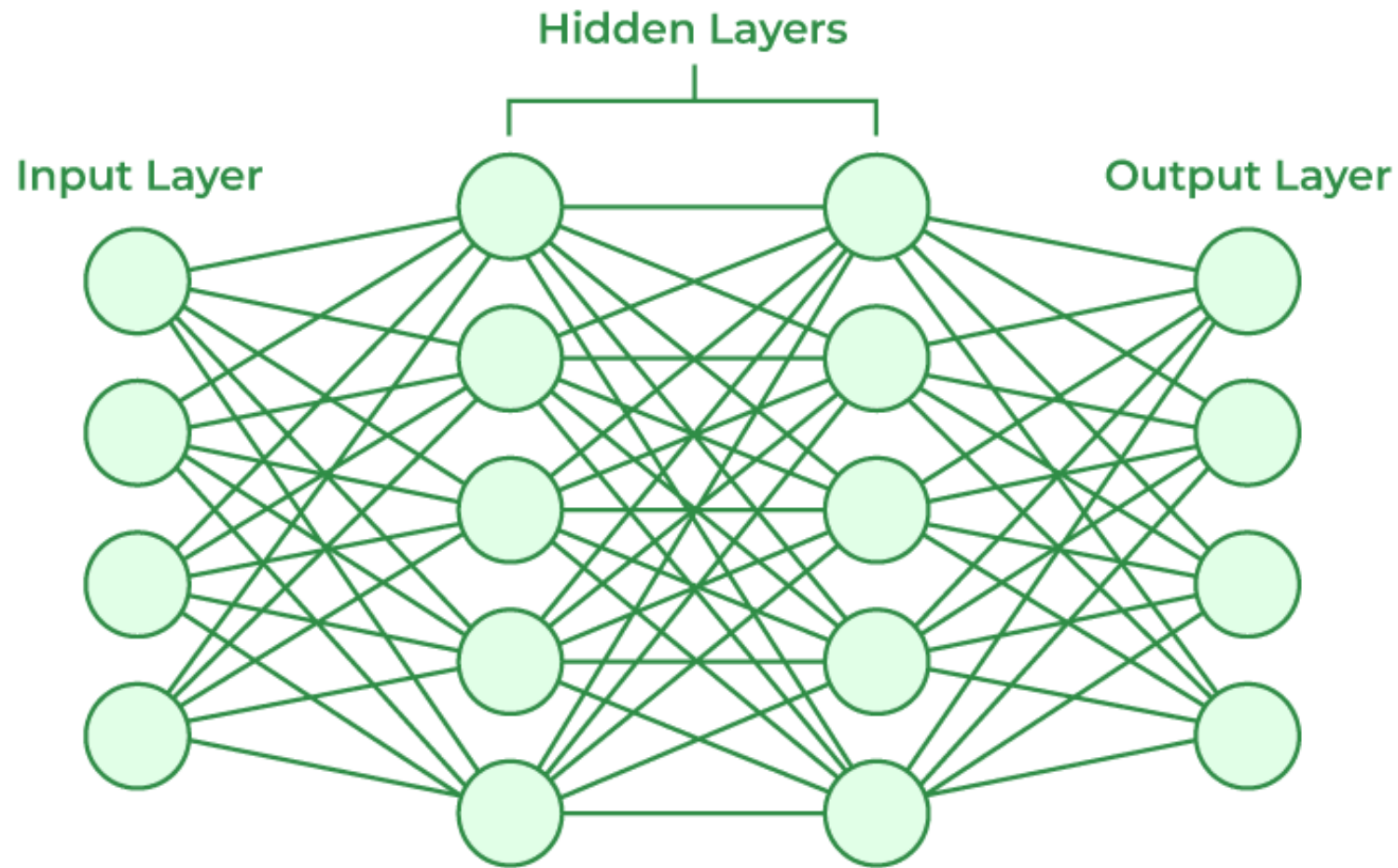


Aarav



Shaun

Neural networks = a class of machine learning models designed specifically to detect complex patterns that simpler methods miss



Motivation

Linear models (*Ridge, OLS, Lasso, Elastic Net*)

These assume each characteristic contributes independently and additively to predicted return:

$$\hat{y} = \beta_1 \times \text{momentum} + \beta_2 \times \text{size} + \beta_3 \times \text{volatility} + \dots$$

Nonlinear models (*neural networks*)

Stock returns are driven by interactions between characteristics; the same variable can mean completely different things depending on context:

$$\hat{y} = f(\text{momentum, size, volatility, ...})$$

where f captures complex combinations

Momentum + size

High momentum predicts strong future returns for small stocks — but barely works for large stocks. A linear model applies the same momentum coefficient to both.

Volatility + quality

High volatility usually predicts lower returns — but for high-quality firms, volatile periods are buying opportunities. The low-volatility anomaly disappears conditionally.

Value + market conditions

Value stocks outperform when aggregate book-to-market is high (cheap market). The same value signal has opposite implications in expensive vs cheap markets.

+ *manageable dimensionality*

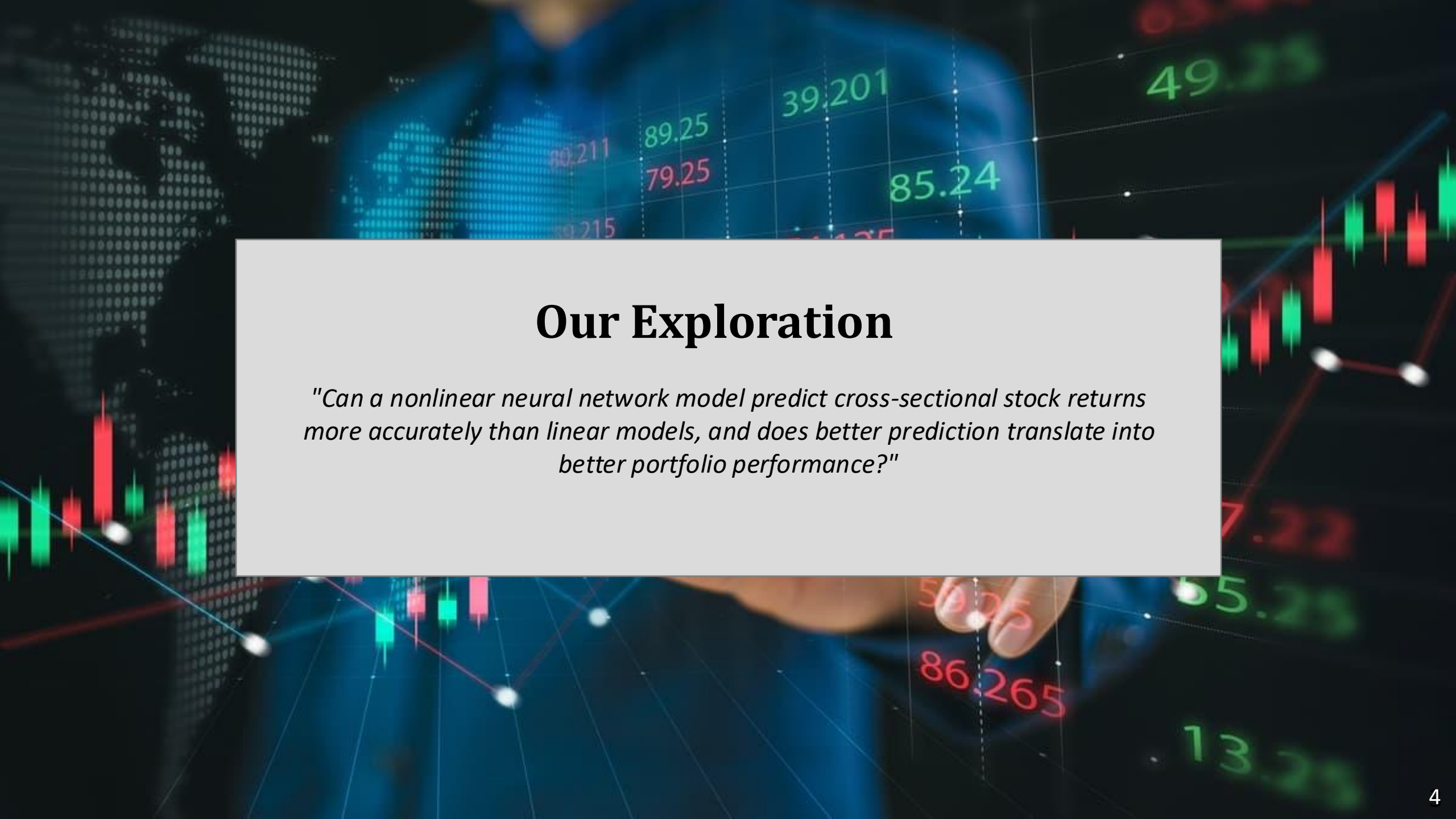
WHAT *Gu Kelly Xiu* (2020) FOUND

0.61

Sharpe ratio — leading OLS regression benchmark

1.35

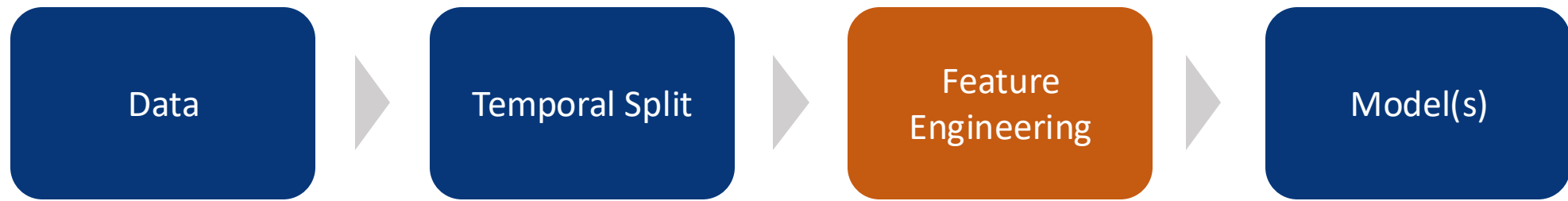
Sharpe ratio — neural network long-short portfolio

The background of the slide is a blurred image of a financial market. It features various stock charts, including candlestick charts and line graphs, with red and green colors indicating price movements. Numerical data points are visible in green and red, such as 89.25, 79.25, 39.201, 85.24, 49.25, 80.211, 215, 59.25, 86.265, 13.25, and 7.22. The overall theme is financial analysis and stock market performance.

Our Exploration

"Can a nonlinear neural network model predict cross-sectional stock returns more accurately than linear models, and does better prediction translate into better portfolio performance?"

Pipeline



Feature Engineering & Preprocessing Pipeline

From 202 raw characteristics to 87 orthogonal predictors — 6-step pipeline with no look-ahead discipline

1	Raw Data 202 columns	<i>JKP US equity dataset</i>
2	Missing Filter (<50%) 202 → 185 features	<i>17 features with sparse coverage dropped</i>
3	Selective Log-Transform 185 features (170 transformed)	<i>$\text{sign}(x) \cdot \log(1+ x)$ for $\text{skew} > 1.0$ (training only)</i>
4	IC Screening 185 → 140 features	<i>Keep $\text{IC t-stat} \geq 1.5$ (training period only)</i>
5	JKP Theme Grouping 140 → 14 groups	<i>Jensen, Kelly & Pedersen (2023) taxonomy</i>
6	Grouped PCA 14 groups → 87 components	<i>90% variance threshold per group — orthogonal, theme-structured</i>
7	GKX NN3 Ensemble 87 components → Predictions	<i>5-seed ensemble neural network, test period 2019–2024</i>

All preprocessing parameters fitted on 2005–2015 training data only — zero look-ahead into validation (2016–2018) or test (2019–2024).

Step 1: Starting Point

202 columns → 185 features after missing filter

Dataset: Jensen, Kelly & Pedersen (2023)

JKP US equity characteristics database
~3 million stock-month observations
202 raw characteristic columns
Target variable: ret_exc_lead1m (next month excess return)
Periods: Train 2005–2015 | Val 2016–2018 | Test 2019–2024

First Filter: Drop Features with >50% Missing

Applied on training period (2005–2015) only
Features with systematically sparse coverage add noise — not signal
Threshold selected to exclude sector-specific metrics absent from broad universe

Before	After	Dropped
202	185	17
features	features	features

All 17 dropped features have >50% missing in training: mostly R&D and rare dividend metrics (rd_me, rd_sale, rd5_at, divspc1m_me)

Examples of Dropped Features

rd_me – Sparse, mostly missing outside R&D Intensive Sectors
R&D to market equity

rd_sale – Sparse. Majority firms do not report R&D
R&D to sales ratio

rd5_at – Rarely Available
5-year R&D intensity

divspc1m_me – Special Dividends are rarely paid. Effectively zero for most firms.
Special dividends

Step 2: Selective Log Transform

185 features → 170 log-transformed

Why Log-Transform?

Many financial characteristics have extreme right tails
Skewed inputs hurt NN training:

- Winsorisation bounds become unreliable under heavy skew
- Gradient updates dominated by outlier-driven magnitudes

Solution: log-transform only skewed features ($|skew| > 1.0$)

Transform Method

$$f(x) = \text{sign}(x) \cdot \log(1 + |x|)$$

- $\text{sign}(x)$ preserves direction for negative values
- $\log(1 + |x|)$ compresses the tail, preserving small values near zero
- NaN-safe: missing values remain missing after transform
- Monotone → IC (Spearman rank) is invariant — screening results unchanged

Result

Features assessed: 185 | Log-transformed: 170 |
Unchanged: 15

Top 10 Most Skewed Features (training 2005–2015)

Feature	Skewness
ret_1_0	1,224
rmax1_21d	1,178
ivol_ff3_21d	1,168
ivol_capm_21d	1,162
rvol_21d	1,154
rmax5_21d	1,083
chcsho_1m	1,060
ivol_hxz4_21d	1,008
tvol	918
chcsho_3m	897

All transform decisions made on training data only — applied identically to validation and test.

Skew

$$E[(X - \text{mean})^3 / \text{std}^3]$$

Step 3: IC Screening

185 → 140 features kept

What is IC?

IC (Information Coefficient) = monthly cross-sectional rank correlation of each feature with next-month returns

IC t-stat = $\text{mean IC} / \text{std(IC)} \times \text{sqrt(T)}$ where T = months with ≥50 stocks

Threshold: $|\text{IC t-stat}| \geq 1.5 \rightarrow \text{keep}$ | below $\rightarrow \text{drop}$.

Why 1.5? → A complex model can still make use of a weak signal compared to a linear model

IC Screening Summary	
Training period	2005–2015 (132 months)
Min stocks per month	50
Threshold	$ \text{IC t-stat} \geq 1.5$
Features in	185
Features kept	140
Features dropped	45
Sign consistency (train vs val)	92.1%
Mean $ \text{val t-stat} $ (kept)	2.46

- For each feature (say, profitability), every month you rank all stocks from lowest to highest on that feature. Then you check: did the stocks you ranked highly actually earn higher returns next month?
- The **IC** is just the correlation between your ranking and what actually happened. If your ranking of stocks by profitability consistently matches next month's return ranking, the IC is high — the feature is useful. If it's random noise, IC is near zero
- The **t-stat** just asks: "is this consistency reliable, or did it happen a few lucky months?" A feature that works 6 out of 10 months is more trustworthy than one that had one great month and was flat otherwise.

Top 10 Features by $ \text{IC t-stat} $			
Rank	Feature	IC t-stat	Theme
1	ocf_at	+15.6	Profitability
2	niq_at	+14.1	Profitability
3	niq_be	+13.3	Profitability
4	cop_at	+12.9	Profitability
5	op_at	+12.6	Profitability
6	ope_be	+12.4	Profitability
7	qmj_prof	+12.1	Quality
8	cop_atl1	+11.6	Profitability
9	ebit_bev	+11.2	Value
10	o_score	-10.8	Quality

Profitability and quality signals dominate the top 10. The first 8 are all current earnings/cash flow measures — consistent with their economic logic.

Step 4: Feature Grouping into 14 JKP Themes

140 features assigned to 14 economic groups for grouped PCA

Why group before PCA?

Global PCA mixes momentum with volatility with value — components lose economic meaning. Grouped PCA keeps components interpretable: momentum_PC1, value_PC1, etc.

Source: Jensen, Kelly & Pedersen (2023) 13 themes + catch-all 'other'

Group	Raw Feats	IC Kept	Economic Interpretation
Momentum	22	~20	Past return continuation across horizons
Short-Term Reversal	1	0 → skipped	1-month price reversal — dropped by IC screening
Seasonality	6	~5	Calendar-based return patterns
Value	17	~14	Price relative to accounting fundamentals
Profitability	16	~14	Current earnings/cash flow quality
Profit Growth	15	~12	Changes in profitability over time
Investment	18	~10	Asset growth and capital expenditure
Accruals	14	~11	Non-cash earnings components
Debt Issuance	14	~10	Share/debt issuance activity
Leverage	6	~3	Capital structure and debt ratios
Low Risk	23	~20	Volatility, beta, and downside risk
Quality	16	~14	Composite firm quality scores
Size & Liquidity	13	~9	Trading activity and market microstructure
Other	4	~3	Miscellaneous — no clean JKP fit
TOTAL	185	140	

Step 5: Grouped PCA Methodology

140 features → 87 orthogonal components

Algorithm (per group)	
1	Intersect group members with IC-selected 140 features
2	0 features survive → skip group (e.g. short_term_reversal dropped by IC)
3	1 feature survives → pass through as-is (no PCA needed)
4	≥ 2 features survive → fit full PCA on training data
5	Find k = smallest # components where cumulative var ≥ 90%
6	Fit final PCA(k) on training data, transform train/val/test

No Look-Ahead — Enforced at Every Stage	
Skewness values (log-transform decision)	Training only
Winsorisation bounds [1st, 99th pct]	Training only
Median imputation values	Training only
Z-score (mean, std)	Training only
IC t-statistics (feature selection)	Training only
PCA eigenvectors	Training only

Why 90% Variance Threshold?

- Retains dominant variation within each economic theme
- Discards noise dimensions — components explaining <10% collectively within a group
- Higher than this would mean diminishing returns

Output Naming Convention

Components named: momentum_PC1, momentum_PC2, value_PC1, ...

Each component is interpretable as a 'direction of variation' within its economic theme — not an opaque global factor

Why PCA?

- We understand that PCA is little good to even being bad for NNs
- But that does not mean it's bad for financial performance
- By reducing noise and overfitting, the overall financial performance increases
- PCA helps in reducing noise and overfitting

Why Not Global PCA?

- 22 momentum signals are highly correlated internally
- Global PCA would let them dominate the first few components
- Other themes (value, quality) underweighted — economically distorted representation
- Grouped PCA allocates components proportionally across all 14 themes

Step 5: Results for Grouped PCA

140 features → 87 orthogonal components

Group	Features In	IC Survivors	Components Out	Var Explained
momentum	22	~20	~3	≥ 90%
short_term_reversal	1	0	—	skipped
seasonality	6	~5	~2	≥ 90%
value	17	~14	~4	≥ 90%
profitability	16	~14	~3	≥ 90%
profit_growth	15	~12	~4	≥ 90%
investment	18	~10	~5	≥ 90%
accruals	14	~11	~4	≥ 90%
debt_issuance	14	~10	~4	≥ 90%
leverage	6	~3	~2	≥ 90%
low_risk	23	~20	~3	≥ 90%
quality	16	~14	~4	≥ 90%
size_liquidity	13	~9	~3	≥ 90%
other	4	~3	~2	≥ 90%
TOTAL	185	140	87	

short_term_reversal (ret_1_0) was dropped by IC screening → its group skipped entirely. The 1-month reversal signal had no consistent predictive power in the training period at our threshold.

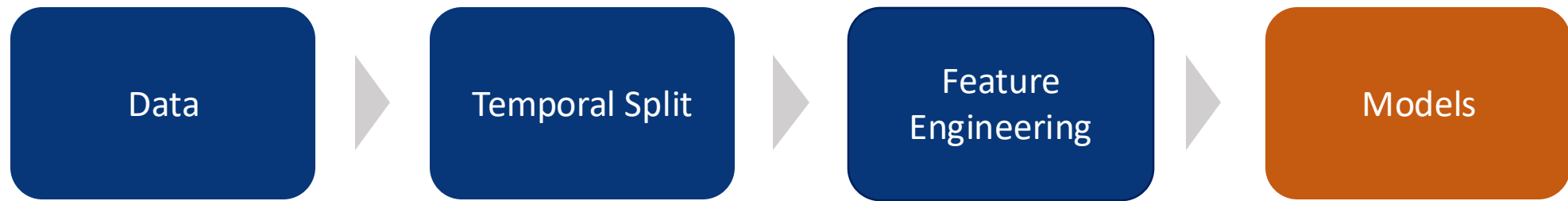
Full Pipeline Summary

All preprocessing parameters fitted on training period 2005–2015 only. Validation 2016–2018 and test 2019–2024 use training-fitted parameters exclusively.

Key: 202 raw features → 87 orthogonal, theme-structured components via a 6-step pipeline with strict no-look-ahead discipline at every stage.

Step	Operation	Description	Input	Output
1	Raw data	Full JKP US equity characteristics	202 cols	202 features
1	Missing filter	Drop features with >50% missing (training)	202 features	185 features
2	Log-transform	$\text{sign}(x) \cdot \log(1 + x)$ for $ \text{skew} > 1.0$ (training only)	185 features	185 features (170 transformed)
3	IC screening	Keep $ \text{IC t-stat} \geq 1.5$ (training only)	185 features	140 features
4	Theme grouping	Assign to 14 JKP economic themes	140 features	14 groups
5	Grouped PCA	90% variance threshold per group (training only)	140 features	87 components
Model(s)	GKX NN3	5-seed ensemble neural network	87 components	Return predictions

Pipeline



1. Ensemble model (based on paper implementing GKX on korean stocks)
 - Train 5 different NNs initialized with different random seeds on stock data to make predictions (MSE loss function).
 - To predict, simply generate all 5 predictions and take the mean.

Better performance of non linear models

We attribute the improvement in performance of non-linear models to the extended representative power they have to model higher order relationships.

The model is actually quite simple:

Each nn only consists of 3 linear layers with activation

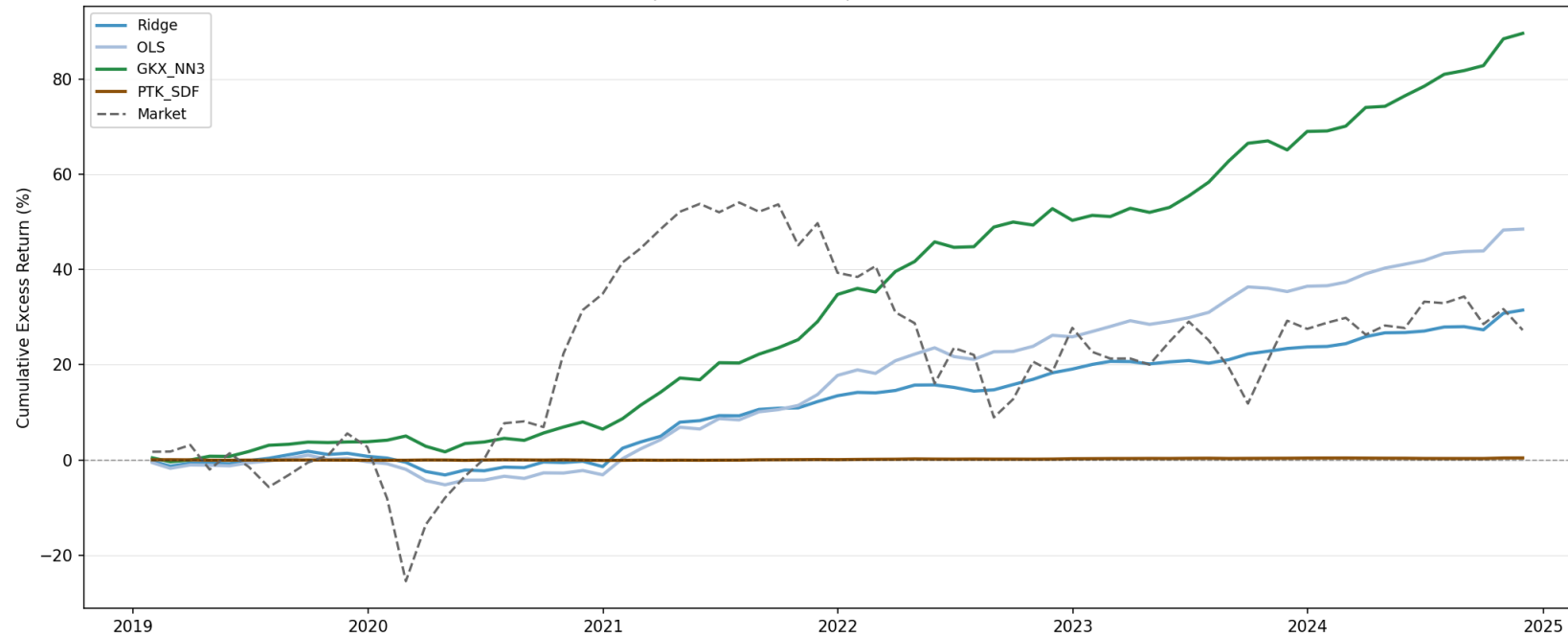
This increase in predictive power leads to much better results, and can naturally be better optimized by doing hyperparameter tuning, different architectures, or data preprocessing

At the same time, we also see very poor performance with the ptk network which also reflects the potential drawback of NN for overfitting

Results

Results: jkp_data.parquet

Model Comparison: Out-of-Sample Performance (2019-2024)



Model	OOS R^2	Sharpe Ratio
Ridge	-0.0008	1.53
Historical Average	0	0
OLS	0.02020	1.72
Ensemble Network	-0.003699	2.43
PTK-sdf	n/a	0.62

Discussion on the results

- OLS outperforms ridge regression, suggesting that cutting features actually hurts it.
- We can see a significant improvement between the linear (OLS, ridge) baselines and the non-linear model in terms of Sharpe Ratio (~0.7 increase)
- We see an increase in Sharpe despite having lower OOS R^2 between OLS and ensemble

Sharpe vs R^2 discrepancy

- Intuitively, we expect a strong correlation between OOS r^2 and sharpe, a higher R^2 typically results in better predictions hence leading to more optimized portfolio weights and thus higher returns.
- Interestingly, when we compare OLS and the non linear model, we see that the ensemble has higher Sharpe but much lower OOS R^2
- We believe that this is because the nn misses the mark but is correct directionally. We observe that it has the highest IC mean among all the models. Therefore, though it makes poor predictions due to magnitude, it is accurate in rating the stocks relative to each other which translates to better portfolio construction

Models (documented attempt at optimizing returns)

1. PTK model (based on large and deep factor models)

- Train NN on custom loss function (MSRR) which directly optimizes portfolio returns

$$\mathcal{L}_t = \frac{(1 - f_t^\top R_{t+1})^2}{N_t}.$$

- Later on uses ridge regression on gradients of the nn to price the factors.

Overfitting hypothesis

We hypothesize that the reason for the PTK network producing bad results is owing to **overfitting**. The model architecture was inspired by a paper "Large and Deep Factor Models" by Bryan Kelly, and in their results they are able to produce sharpe ratios >3 .

- We attribute the differences to dataset difference. In particular, their dataset ranges from 1963 –2024 vs the compressed one from 2005-2024. This leads us to suspect potential overfitting problem given that the data is much smaller (ie, not enough data to fit, aggressively increasing learning rate then overshoots)

Our work: We test this hypothesis by retraining the model repeatedly via hyperparameter tuning. We see that tweaks in lr and ridge penalty results in quite significant changes in sharpe indicating very unstable learning which supports this argument.

Although not consistently, expanding data set (ie, training on full vs slim) results in worse sharpe which also supports this hypothesis.

PTK model: Instability

Dataset	15 epochs, 1e-3 lr, 1e-4 rp	10 epochs, 1e-4 lr, 1e-3 rp	20 epochs, 1e-4 lr, 1e-6 rp	20 epochs, 1e-3 lr, 1e-3 rp	20 epochs, 1e-3 lr, 1e-3 rp
jkp_data_slim	n/a	0.17	0.62	-0.2	n/a
jkp_data	0.6	0.38	-0.11	-0.65	0.9

Conclusion and extensions

- Limitations:
 - Limited model explainability
 - Time consuming : each run of a very simple model takes 3 hours to train
 - Requires careful tuning
- Extensions:
 - Explore optimizing the pricing pipeline
 - PTK tried but unfortunately cannot replicate results
 - Auto encoder approach by Gu Kelly Xiu:
<https://www.sciencedirect.com/science/article/pii/S0304407620301998>
 - Alternative architectures (ie. Random forests)